

به نام خدا

نیما صهبی - ۴۰۴۴۶۳۷۱۰

تاریخ: ۱۴۰۵/۵/۲۰

گزارش نهایی پروژه درس برنامه‌نویسی پیشرفته

۱. مقدمه و تحلیل نیازمندی‌ها (Requirement Analysis)

هدف از این پروژه، طراحی و پیاده‌سازی بازی کلاسیک Pac-Man می‌باشد که با اصول شی‌گرایی (OOP) و زبان Java نوشته شده است. این پروژه بر اساس الگو معماری MVC دسته‌بندی شده است. با توجه به این لایه بندی:

- پوشه **Model** مربوط به اطلاعات بازی، پایگاه داده، صدا و موجودیت های بازی از جمله Pacman و Ghost که فرزند کلاس Block هستند.
- پوشه **View** مربوط به اجزای رابط گرافیکی بازی هستند که شامل دکمه‌ها، نوشته‌ها، عکس‌ها و خود پنجره اصلی می‌باشند.
- پوشه **Controller** منطق و قوانین بازی را شکل می‌دهد. این پوشه، کلاس اصلی اجرای بازی، برخورد بلاک‌ها، حرکت درست موجودیت های بازی و دریافت رخدادها از جمله دکمه‌های کیبورد را کنترل می‌کند.
- پوشه **assets** تصاویر و صداها می‌باشد که به بازی و آیکون بازی را نگهداری می‌کند.
- پوشه **font** دارای یک فایل با پسوند .ttf است که مربوط به نوشته‌های بازی می‌باشد و نصب آن الزامی است.

- پوشه **lib** درایور پایگاه داده **sqlite** را در خود دارد.

ابعاد بازی 608 x 704 است. این ابعاد بر اساس تصاویر پکمن، ارواح و دیوار است که همه‌ی این تصاویر ابعاد 32x32 پیکسل را دارند. بازی دارای ۲۲ سطر و ۱۹ ستون است که هر کدام ۳۲ پیکسل دارد. سرعت پکمن ۸ پیکسل بر فریم است.

۲. توضیح کلاس‌ها

Block: این کلاس اطلاعات مربوط به دیوارها و خود پکمن و ارواح را مدیریت می‌کند. در متد سازنده این کلاس، پارامترهای `x, y, width, height, img` دریافت می‌شوند؛ به عبارتی بلاک درست شده، ویژگی مختصات، ابعاد آن و تصویر که از نوع `Image` می‌باشد مقداردهی اولیه می‌شوند. کلاس بلاک، کلاس پدر برای `Pacman` و `Ghost` است. این کلاس دارای توابع متعددی می‌باشد که سرعت را بر اساس جهت تنظیم می‌کند، جهت فعلی را بروزرسانی می‌کند و توابع `getter` و `setter` برای `property` های لازم تعریف شده است.

Pacman: کلاس پکمن فرزند کلاس `Block` است. در متد سازنده این کلاس پارامترهای کلاس پدر هم مقداردهی اولیه می‌شوند اما مختصات آن در دو متغیر دیگر ذخیره می‌شود تا در صورت نیاز از مختصات اولیه پکمن استفاده کرد و علاوه بر آن دو پارامتر دیگر از نوع کلاس `ScoreManager` و `Sound` وجود دارند که در همان متد سازنده کلاس پکمن مقدار دهی می‌شوند. این کلاس تابعی دارد که برخورد خود را با دانه‌های بازی بررسی می‌کند و اگر برخورد صورت بگیرد، آن دانه از لیست دانه‌ها حذف می‌شود و ۱۰ امتیاز به امتیازات بازیکن اضافه می‌شود و صدای مربوطه پخش می‌شود. توابع دیگر عبارتند از: بررسی حرکت درست و منطقی پکمن، برگشت پکمن به مختصات اولیه و تلیپورت شدن در سطر نوزدهم.

Ghost: این کلاس هم فرزند کلاس Block است. دارای یک array چهارتایی از نوع char می باشد که چهار جهت اصلی به اختصار نوشته شده است و یک متغیر از کلاس Random دارد. متد سازنده این کلاس هم کار سازنده کلاس Block را انجام می دهد و تابع تولید شانسی جهت را فراخوانی می کند که در آن یک عدد بین ۰ تا ۳ به طور شانسی انتخاب می شود تا ایندکس array جهت ها مشخص شود در نتیجه جهت برای ارواح به صورت شانسی انتخاب می شود. توابع دیگر این کلاس ریست کردن مختصات ارواح، بررسی حرکت و برخورد با دیوار است.

Map: این کلاس نقشه بازی را کنترل می کند. دارای یک ویژگی آرایه از جنس String دارد که هر عضو آن یک رشته است که سطر را مشخص می کند و عضوهای بعدی سطرهای بعدی را. **Pellet:** این کلاس از کلاس Block ارث بری می کند و متد سازنده کلاس Block را در خود فراخوانی می کند.

SocreManager: این کلاس، جان و امتیاز کاربر را مدیریت می کند. دارای توابع getter و setter است و توابع دیگر آن، ریست کردن جان و امتیاز، کم کردن امتیاز و جان می باشد.

Database: این کلاس مربوط به پایگاه داده می باشد و متغیری از نوع رشته دارد که روش اتصال sqlite و نام فایل پایگاه داده را مشخص می کند. تابعی به نام connect دارد که پایگاه داده را به جاوا متصل می کند. تابع دیگر آن createTable است که id و نام بازیکن از جنس رشته و امتیاز بازیکن از جنس int را داراست. توابعی دیگر از جمله addReocrd, highScore و highScoreName و disconnect را دارد.

Sound: این کلاس صدای بازی را کنترل می کند. دارای متغیرهایی از نوع Clip دراد که در واقع فایل صوتی را در خود نگه می دارند. تابعی به نام loadClip وجود دارد که یک رشته به عنوان

آرگومان می‌گیرد و به دنبال آن فایل صوتی می‌رود با آدرسی که داده شده، فایل صوتی باز می‌شود و به کلیپ داده می‌شود و شی از کلاس **Clip** را بر می‌گرداند. این تابع برای همه صداها در متد سازنده فراخوانی شده تا آماده پخش باشند. تابعی برای پخش صدا وجود دارد که متغیری از نوع **Clip** می‌گیرد و آن را از اول پخش می‌کند و این تابع برای همه تابع های مجزا برای هر صدا فراخوانی شده است.

MyFrame: این کلاس قاب بازی را طراحی، و از کلاس **JFrame** ارث‌بری می‌کند. در متد سازنده آن تیترا قاب، آیکون و مختصات آن تعریف می‌شود. همچنین اندازه قاب غیرقابل تغییر است و با فشردن دکمه X برنامه بسته می‌شود.

MyPanel: این کلاس رابط گرافیکی بازی و اجزای آن را مدیریت می‌کند و از کلاس **JPanel** ارث‌بری می‌کند و کلاس **ActionListener** را پیاده‌سازی می‌کند. در این کلاس اندازه پنجره، دکمه‌ها، نوشته‌ها، تکست‌فیلد، تصاویر، کیبورد، صداها، دیوارها، ارواح، پک‌من، اطلاعات بازی، نقشه، حلقه، حرکت تعریف می‌شوند و بیشترشان در متد سازنده مقداره‌ی می‌شوند. توابع این کلاس مربوط به رسم اجزای بازی می‌شوند (**paintComponent, draw**). همینطور در تابع **update** حرکت تمام موجودیت‌های بازی که می‌توانند حرکت کنند، بررسی و اعمال می‌شود. در تابع **actionPerformed** دکمه‌ای که کاربر می‌فشارد بررسی می‌شود و با توجه به آن، تغییرات مورد نظر انجام می‌شود.

MyButton: این کلاس از **JButton** ارث‌بری می‌کند. در متد سازنده آن یک رشته برای متن دکمه، یک عدد صحیح برای سایز فونت، چهار عدد صحیح برای مختصات و ابعاد و یک پنل برای ثبت رخدادها برای دکمه به عنوان آرگومان داده می‌شود. دو تابع هم برای آشکار کردن و پنهان کردن دکمه‌ها نوشته شده است.

MyLabel: این کلاس از JLabel ارث‌بری می‌کند. در متد سازنده آن یک رشته برای متن

نوشته، یک عدد صحیح برای سایز نوشته، یک رنگ برای انتخاب رنگ خود نوشته، چهار عدد صحیح

برای مختصات و ابعاد به عنوان آرگومان داده می‌شود. دو تابع برای پنهان کردن و آشکار کردن

نوشته‌ها وجود دارد که به طور دلخواه نوشته‌های مورد نظر به عنوان آرگومان داده می‌شوند. و

همینطور نوشته‌ها در مرکز مستطیل فرضی قرار دارند.

MyImage: این کلاس تصاویر بازی را برعهده دارد. ۹ متغیر از نوع Image ساخته شده‌اند که

شامل ۴ تا روح، دیوار و ۴ تا تصویر از پک‌من در جهت‌های مختلف می‌باشد. توابعی برای بارگذاری

کردن تصاویر برای ارواح، پک‌من و دیوار اختصاص داده شده است. همینطور تابعی برای پک‌من با

توجه به جهت حرکت آن ساخته شده تا تصویر آن با جهت حرکت هماهنگ باشد. توابع getter هم

ساخته شده‌اند.

MyTextField: این کلاس از JTextField ارث‌بری می‌کند. متد سازنده یک رشته برای نوشته

پیش‌فرض، چهار عدد صحیح برای مختصات و ابعاد آن به عنوان آرگومان دریافت می‌کند. نوشته‌های

داخل تکست‌فیلد وسط‌چین هستند.

Collision: این کلاس برخورد بلاک‌ها را برعهده دارد. تابع checkCollision دو عدد صحیح

برای مختصات و دو شی از کلاس بلاک را دریافت می‌کند. در این تابع دو مستطیل ساخته می‌شود با

مختصات و ابعاد بلاک‌ها. سپس برخورد آنها بررسی می‌شود. همین تابع overload شده ولی فقط

دو بلاک به عنوان آرگومان دریافت می‌کند. تابعی برای تغییر جهت درست شده تا بررسی کند تغییر

جهت میسر است یا خیر. آخرین تابع هم مربوط به تغییر جهت از جنبه و دید جهت است که جهت به

عنوان آرگومان داده می‌شود و براساس آن تابع قبلی فراخوانی می‌شود و امکان حرکت در جهت

درخواستی را بررسی و اعمال می‌کند.

Game: این کلاس شرایط برد و باخت را کنترل می‌کند. چهار متغیر از نوع Boolean وجود دارد که شرایط بازی را مدیریت می‌کنند. در متد سازنده این کلاس، دو متغیر از جنس Sound و **ScoreManager:** مقداردهی می‌شوند. دو تابع برای بررسی برد و باخت وجود دارد. تابع برد

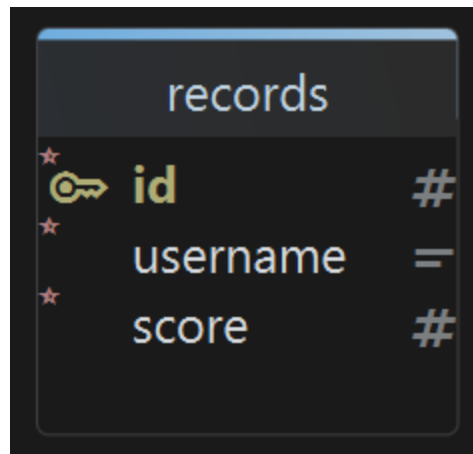
اگر همه دانه خورده شده باشند ۵۰۰ امتیاز اضافه می‌کند، صدای مربوطه را پخش و true بر می‌گرداند. تابع باخت اگر جان بازیکن صفر باشد آنگاه صدای مربوطه را پخش و true بر می‌گرداند.

Keyboard: این کلاس در متد سازنده خود، پک‌من، دیوارها، تصویر و اطلاعات را مقداردهی می‌کند. این کلاس KeyListener را پیاده‌سازی می‌کند پس باید سه متد مربوطه override شوند و کلاس keyPressed برای مدیریت دکمه‌های کیبورد انتخاب شده است. که اگر arrow keys باشند جهت مناسب در نظر گرفته می‌شود و اگر حرکت منطقی باشد انجام می‌شود و یک امتیاز از بازیکن کم می‌شود.

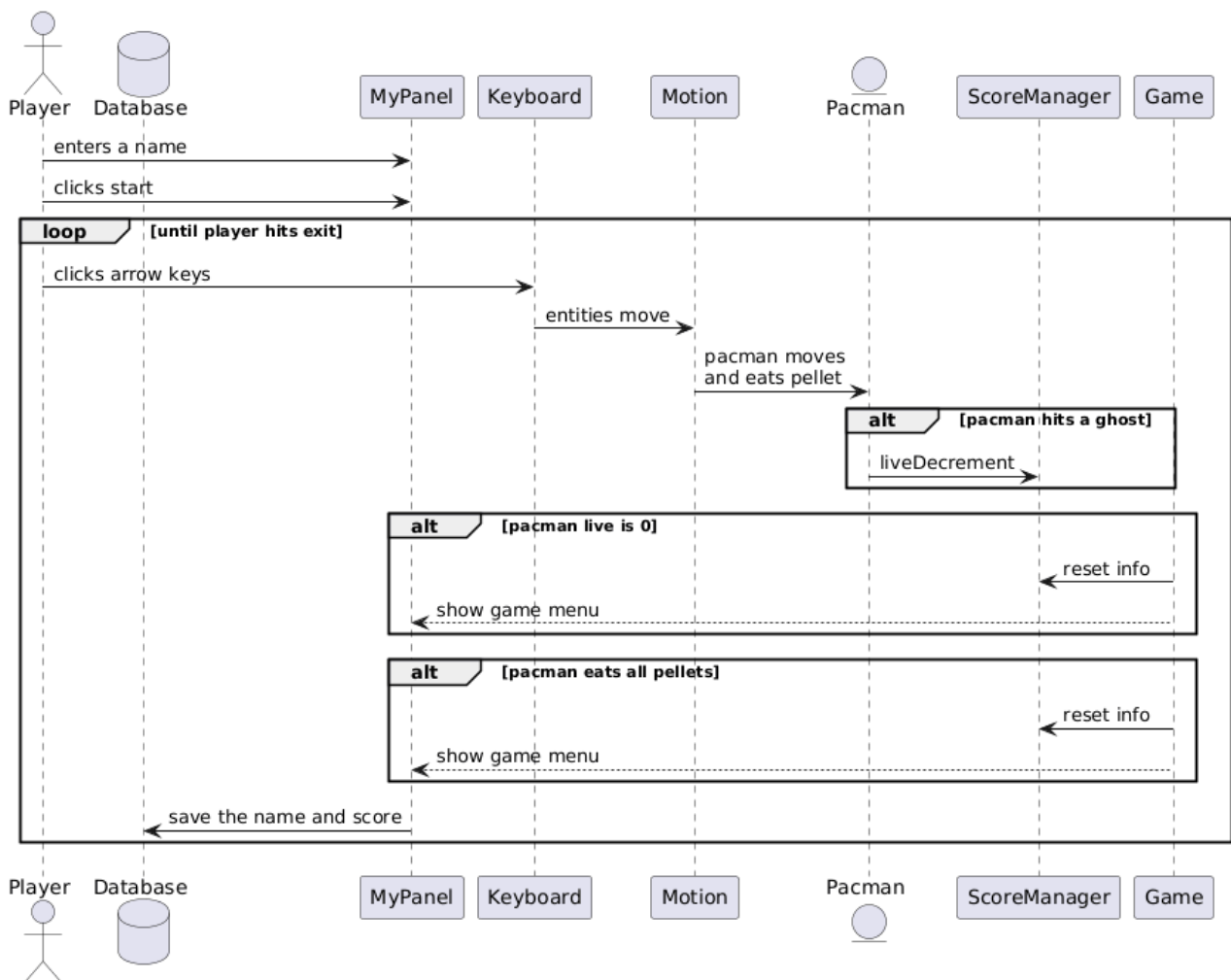
Motion: این کلاس مربوط به حرکت پک‌من و ارواح می‌شود. در متد سازنده خود پک‌من، دیوارها، ارواح، اطلاعات بازی و اینفو مقداردهی می‌شوند. در تابع move در صورتی که بازی شروع شده باشد حرکت‌های مربوط به پک‌من و ارواح انجام می‌شود و توابع مربوط به پک‌من فراخوانی می‌شود. اگر به روحی برخورد داشته باشد پک‌من و ارواح به جای اول خود بر می‌گردند.

Main: در این کلاس بازی اجرا می‌شود و در آن ظاهر پیش‌فرض اجزای swing، به Cross Platform Look and Feel تغییر می‌کند تا ظاهر بازی وابسته به سیستم عامل نباشد. این کلاس یک شی از کلاس Database می‌سازد و آن را متصل می‌کند و بعد جدول می‌سازد. سپس قاب و پنل ساخته می‌شوند و پنل به قاب اضافه می‌شود. در آخر با متد setVisible بازی آشکار می‌شود.

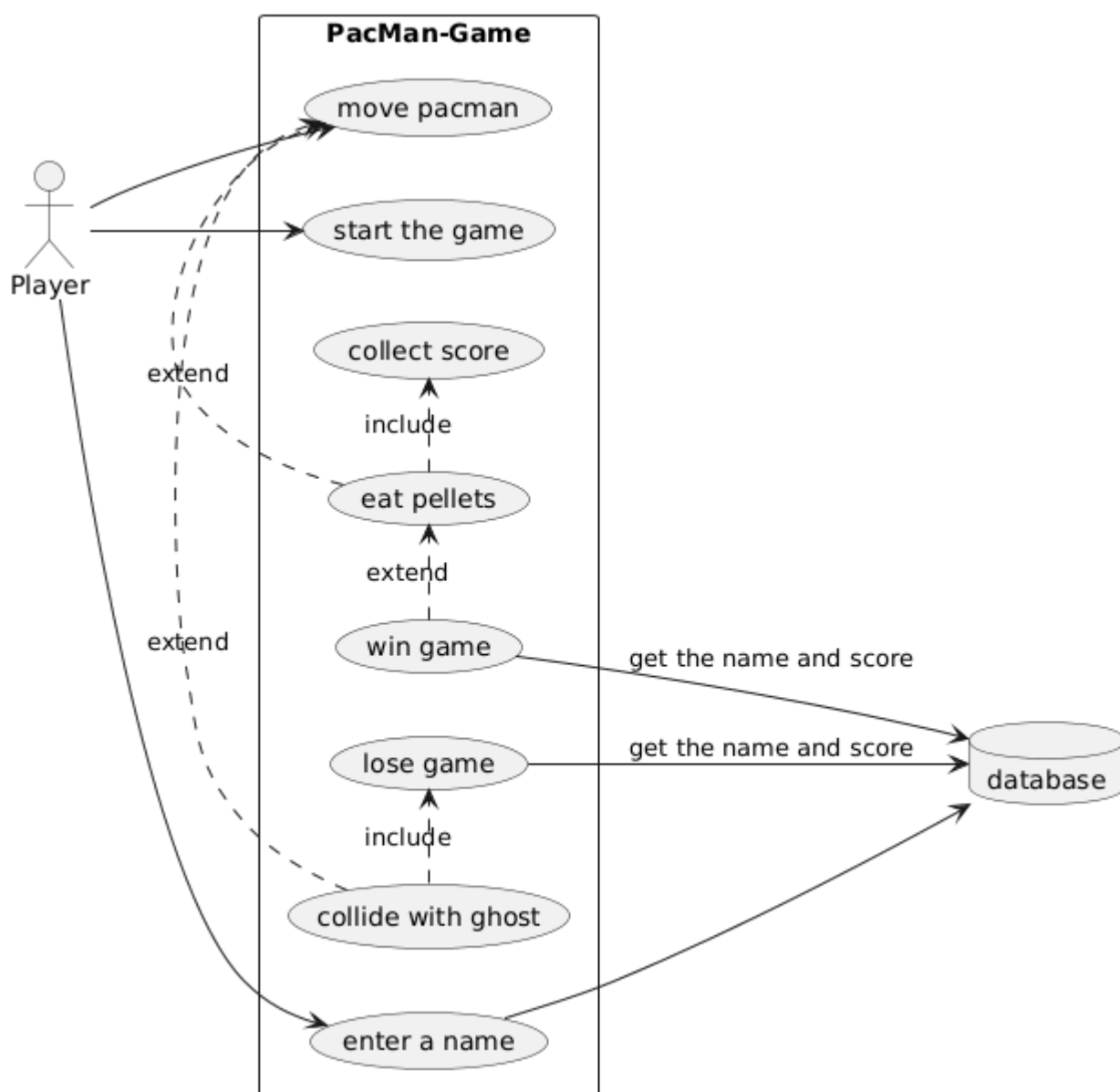
نمودارها (Diagrams):



نمودار بالا، مربوط به ER برای پایگاه داده sqlite می‌باشد. که سه ستون id (primary key)، username از نوع varchar و score از نوع int را دارد.



نمودار فوق، نمودار توالی (Sequence Diagram) است که ترتیب مراحل زمانی نمایش داده شده است.



نمودار فوق، نمودار موارد کاربردی (Use Case Diagram) است. اتفاقات و رخ دادهای مهم بازی و وابستگی ها در این نمودار مشخص شده است.

ضمیمه: چالش‌های پیش رو و نحوه حل آنها و استفاده از هوش مصنوعی

حرکت درست پک‌من: بزرگترین چالش من در طول توسعه پروژه نحوه پیاده‌سازی حرکت پک‌من بود. من ابتدا حرکت پک‌من رو به این صورت تعریف کردم که کاربر با فشردن دکمه جهت مشخص می‌شود و جهت قبلی ذخیره می‌شود و با توجه به آن جهت، سرعت مولفه‌های x, y سرعت مشخص می‌شود و من با اضافه کردن سرعت به مختصات پک‌من اگر برخوردی صورت بگیرد همان سرعت رو از مختصات کم کن تا به مختصات قبلی برگرد و به حرکت قبلی ادامه دهد. این روش باعث شد تا پک‌من حین حرکت صحیح اگر به جهتی خواست حرکت کند که دیوار بود جهت قبلی را به جهت فعلی تبدیل کند، تا این کار منع شود اما باگ بزرگ این روش این بود که اگر خود پک‌من به انتهای مسیر می‌رسید و دیوار بود جهت قبلی به جهت فعلی تبدیل می‌شد و اگر امکان حرکت بود، حرکت ناخواسته رخ می‌داد. من برای حل این مشکل از هوش مصنوعی ایده گرفتم اما برای من مفید نبود و نتوانستم استفاده بنابراین تابع بررسی حرکت را به چند تابع تبدیل کردم تا قدم به قدم حرکت بررسی شود. که با فشردن کلید تابع مربوط به جهت صدا زده شود و بعد تابع سرعت اگر برخوردی نبود آنگاه جهت بروزرسانی شود و بعد حرکت کند.

کلمه کلیدی `super`: من در اوایل کد در متد `paintComponent` از

`super.paintComponent(g)` استفاده کردم اما در متد سازنده `Pacman` از `super()`

استفاده کردم اما پیش از آن تفاوت این دو رو نمی‌دونستم به خاطر همین از `ChatGPT` خواستم

بهم توضیح بده تفاوتشون رو.

شناسایی برخورد: در کلاس Collision باید تابعی می‌نوشتیم که برخورد دو بلاک رو شناسایی

کند. به صورتی دستی نمی‌خواستیم انجام بدم و به دنبال یه تابع از پیش ساخته بودم که از

کتابخانه‌های جاوا استفاده کنم. به خاطر همین از ChatGPT پرسیدم آیا همچین تابع آماده‌ای

هست که من بتونم برخورد رو شناسایی کنم و آن هم بهم جواب داد.

تار کردن Game Menu: هدف من این بود که قبل شروع بازی و زدن دکمه start پس‌زمینه

بازی طوری باشه که نقشه و ارواح و پک‌من رو بشه به طور تار دید اما نمی‌دونستم چطور این کار رو

انجام بدم و از ChatGPT پرسیدم و بعد متوجه شدم به چه شکل باید انجام بدم.

پوشه‌بندی پروژه: من در مورد الگوی معماری MVC اطلاعات کافی رو نداشتم، پس از

ChatGPT پرسیدم و خواستم بهم توضیح بده که هر پوشه چه مسئولیت‌هایی رو به عهده داره و

تقسیم بندی به چه شکل هست. بعد از جواب و تقسیم بندی من یه سری شک‌ها داشتم که آیا کلاس

تو پوشه درست هست یا نه، پس دوباره پرسیدم و راهنمایی گرفتم.

پایگاه داده: من اطلاعی از نحوه وصل کردن sqlite به جاوا نداشتم به خاطر همین تو گوگل سرچ

کردم و فهمیدم باید درایور مربوطه رو تو پروژه قرار بدم اما بعدش رو نمی‌دونستم چیکار کنم به همین

خاطر از ChatGPT پرسیدم که با توضیحات بهم بگه چطور میتونم sqlite رو وصل کنم و بهم

گفت که باید connection رو ببندم به خاطر همون واسه اون هم یه متد نوشتم.

Refactor کردن: اواسط پروژه دیدم کلاس MyPanel خیلی طولانی شده و قسمت های مختلف

باید جدا شه به خاطر همین کلاس های متعددی ساختم تا بار رو از دوش MyPanel کم کنم. و

همین حین بود که دیدم خیلی از جاها از کلمه کلیدی static استفاده کردم، پس از ChatGPT

پرسیدم که بهم توضیح بده و بگه کجاها خوبه استفاده کنم.

UI: برای زیبایی بیشتر منوی بازی من خواستم ظاهر دکمه‌ها، نوشته‌ها و تکست‌فیلد رو تغییر بدم.

برای تغییرات بیشتر از جمله وسط‌چین کردن و در مرکز بودن نوشته‌ها به ChatGPT رجوع کردم و راهنمایی گرفتم.

Sound: من در وهله اول، صداها رو تعریف و لود کردم اما با فراخوانی تابع پخش صدای به خصوص،

هر بار صداها تعریف و لود می‌شد که بهینه نبود و حین اجرای بازی صدا و تصویر هماهنگی کامل نداشتند. با کمک از ChatGPT من در متد سازنده همه صداها رو لود و آماده کردم و در تابع پخش موسیقی شروع صداها رو انجام دادم تا کد عملکرد بهتری در بازی به اجرا بذارد.

داکیومنت و ReadMe: برای نوشتن فایل ReadMe.md و داکیومنت از ChatGPT کمک

گرفته شده اما فقط در جهت مرتب شدن، خوانایی بهتر فایل.